

Relazione Progetto Assembly RISC-V per il Corso di Architetture degli Elaboratori – A.A. 2020/2021 – Gestione di Liste Concatenate

Autrice: Elena Palandri

Indirizzo mail: elena.palandri@stud.unifi.it

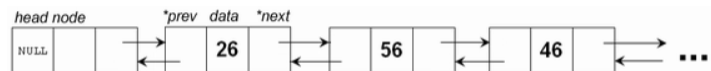
Matricola: 7054433

Data di consegna: 18/02/2022

Liste Concatenate Doppie in RISC-V:

Il progetto di Architetture degli Elaboratori dell'anno accademico 2020-2021 consiste nell'implementazione di un codice RISC-V che possa gestire alcune delle operazioni fondamentali per una lista

concatenata doppia, rappresentata qui a fianco.



Questo tipo di lista, detta anche

doubly linked list è una struttura dati composta da nodi collegati tra loro e ciascuno di essi contiene un riferimento al nodo precedente (prev), a quello successivo (next). Il primo nodo (header) della lista avrà il puntatore prev=NULL, mentre l'ultimo nodo (trailer) avrà il puntatore next=NULL.

Le operazioni richieste sono:

- ADD: Inserimento di un elemento
- DEL: Rimozione di un elemento
- PRINT: Stampa della lista
- SORT: Ordinamento della lista
- REV: Inversione degli elementi della lista

Ogni elemento della lista è composto da 9 byte, suddiviso in:

- PBACK (Byte 0-3): puntatore all'elemento precedente, o contenente 0xFFFFFFFF se primo elemento della lista
- DATA(Byte 4): contiene l'informazione, un carattere ASCII
- PAHEAD (Byte 5-8): puntatore all'elemento successivo, o contenente 0xFFFFFFFF se ultimo elemento della lista

Descrizione:

```
1 #          Progetto Assembly RISC-V per il
2 #          Corso di Architetture degli Elaboratori
3 #          A.A. 2020/2021
4 #          Gestione di Liste Concatenate
5 #          Palandri Elena 7054433
6
7 .data      #dichiaro le stringhe e i caratteri
8 listInput: .string "ADD(1) ~ ADD(a) ~ ADD() ~ ADD(B) ~ ADD ~ ADD(9) ~PRINT~SORT(a)~PRINT~DEL(bb)~DEL(B)~PRINT~REV~PRINT"
9 #listInput: .string "ADD(1) ~ ADD(a) ~ ADD(a) ~ ADD(B) ~ ADD(; ) ~ ADD(9) ~PRINT~SORT~PRINT~DEL(b) ~DEL(B)~PRI~REV~PRINT"
10 tilde: .byte 126
11 space: .byte 32
12
13 .text
14 Main:      #inserisco i vari valori nei registri
15     la s0 listInput
16     lw s1 tilde
17     lw s2 space
18     li a3 0x00010000      #indirizzo base
19     li a5 0x00010000      #indirizzo per LFSR
20     li t3 0               #contatore elementi
```

Nella prima sezione del progetto dichiaro le stringhe e i vari registri.

La parte del .data contiene:

- la lista data in input
- la tilde con codice ASCII 126
- lo spazio con codice ASCII 32

Quella del .text contiene la dichiarazione dei registri e tutte le varie funzioni del programma che elaborano la stringa. I registri dichiarati sono:

- s0, che contiene l'address della testa della stringa data in input
- s1, che contiene il codice ASCII di tilde
- s2, che contiene il codice ASCII dello spazio
- a3, che contiene l'indirizzo base del primo nodo con il quale calcoleremo gli indirizzi dei nodi successivi
- a5, che contiene l'indirizzo per l'LSFR (LinearFeedback Shift Register)
- t3, che contiene un contatore per gli elementi della lista

```

22 WhichOperation: #cerco l'operazione da eseguire
23     add t1 t3 s0      #metto in t1 l'indirizzo attuale
24     lb t2 0(t1)      #metto in t2 l'elemento attuale
25     li t0 65          #metto in t0 il carattere 65=A
26     beq t2 t0 TestAdd #t2=A? Se si' vai a TestAdd
27     li t0 68
28     beq t2 t0 TestDel #t2=D? Se si' vai a TestDel
29     li t0 80
30     beq t2 t0 TestPrint #t2=P? Se si' vai a TestPrint
31     li t0 83
32     beq t2 t0 TestSort #t2=S? Se si' vai a TestSort
33     li t0 82
34     beq t2 t0 TestRev  #t2=R? Se si' vai a TestRev
35     beq t2 zero End    #t2=0? Se si' vai a End, la stringa e' terminata
36     bne t2 s2 NextOperation #se non trovo nessuna corrispondenza cerco un'altra operazione
37
38 NextElement: #scorro all'elemento successivo
39     addi t3 t3 1
40     j WhichOperation
41
42 NextOperation: #cerca l'operazione successiva
43     addi t3 t3 1
44     add t1 t3 s0
45     lb t2 0(t1)
46     beq t2 zero End    #siamo alla fine della lista
47     bne t2 s1 NextOperation #cerco fino a quando non trovo la tilde
48     addi t3 t3 1
49     j WhichOperation  #se la trovo torno a WhichOperation
50

```

WhichOperation è una funzione che permette di cercare quale operazione eseguire. Partendo dall'indirizzo della nostra stringa data in input, calcoliamo il carattere che dobbiamo esaminare sommandogli t3, il contatore degli elementi nella lista che abbiamo trovato. Carichiamo in t2 il byte corrispondente a quel carattere. Carico in t0 il carattere 65 corrispondente alla lettera A. Se t2=t0 allora il comando nella lista potrebbe essere una Add dunque vado a TestAdd. Continuo in questo modo con le varie lettere (D, P, S, R) per identificare i vari comandi. Se t2 è uguale a zero allora andiamo a End poiché la stringa è terminata. Infine controllo che il carattere sia diverso dallo spazio, se lo è vuol dire che non ho trovato corrispondenze e quindi cercherò un'altra operazione.

NextElement permette di scorrere all'elemento successivo della lista. Aggiungo 1 a t3, per aggiornare il contatore dei caratteri analizzati e poi torno in WhichOperation.

NextOperation, come NextElement aggiunge 1 al contatore dei caratteri e come in WhichOperation metto in t1 l'indirizzo del carattere che dobbiamo analizzare sommando il contatore all'indirizzo della testa della stringa data in input. Metto in t2 il byte di quel carattere e controllo che sia uguale a zero per terminare l'esecuzione, che non sia uguale a tilde per cercare un'altra operazione. Se invece il carattere è una tilde allora scorro al carattere successivo della stringa e vado a WhichOperation per cercare di individuare l'operazione che segue la tilde.

```

51 TestOperation:
52     addi a6 a6 1
53     add t1 a6 s0
54     lb a2 0(t1)
55     beq a2 zero WrittenCorrectly      #la lista e' finita e l'operazione e' scritta bene
56     beq a2 s1 WrittenCorrectly        #c'e' la tilde, l'operazione e' scritta bene
57     bne a2 s2 NotWrittenCorrectly     #a meno che non ci sia uno spazio, l'operazione non e' scritta bene
58     j TestOperation
59
60 NotWrittenCorrectly:      #se e' scritta male ritorna 0
61     addi a1 zero 0
62     ret
63
64 WrittenCorrectly:        #se e' scritta bene ritorna 1
65     addi a1 zero 1
66     ret

```

TestOperation verrà utilizzato nei test delle varie operazioni per controllare che siano scritte correttamente. a6 è un contatore degli elementi della lista che viene mandato alla funzione tramite un jal e da esso mi ricavo il carattere nella posizione successiva.

Se è uguale a zero allora la lista è terminata ed è scritta correttamente e vado a WrittenCorrectly che mi ritornerà valore 1. Stessa cosa se termina con tilde. Se invece l'operazione non è uno spazio allora l'operazione non è scritta correttamente dunque salto a NotWrittenCorrectly che mi ritornerà valore 0. Dopo questi controlli torno a TestOperation poiché il carattere analizzato è lo spazio, che è tollerato e non influisce sulla correttezza della scrittura dell'operazione.

Add:

```

68 TestAdd:
69     li t0 68                #carico D e controllo
70     addi t3 t3 1            #in t3 salvo la posizione in cui sono arrivata
71     add t1 t3 s0            #in t1 ho l'indirizzo attuale
72     lb t2 0(t1)
73     beq t2 zero End
74     beq t2 s1 NextElement
75     bne t2 t0 NextOperation
76
77     li t0 68                #carico D e controllo
78     addi t3 t3 1
79     add t1 t3 s0
80     lb t2 0(t1)
81     beq t2 zero End
82     beq t2 s1 NextElement
83     bne t2 t0 NextOperation
84
85     li t0 40                #carico ( e controllo
86     addi t3 t3 1
87     add t1 t3 s0
88     lb t2 0(t1)
89     bne t2 t0 NextOperation
90     addi t3 t3 1
91     add t1 t3 s0
92     lb t2 0(t1)
93     blt t2 s2 NextOperation
94     bgt t2 s1 NextOperation
95     add a0 t2 zero

```

```

97  li t0 41                #carico ) e controllo
98  addi t3 t3 1
99  add t1 t3 s0
100 lb t2 0(t1)
101 beq t2 zero End
102 beq t2 s1 WhichOperation #se trovo una tilde vado a WhichOperation
103 bne t2 t0 NextOperation
104
105 add a6 t3 zero          #inizializzo un contatore per capire a che punto della lista sono
106                          #salvo t3 in a6, il che vuol dire quando faccio la funzione uso a6 e so dove sono arrivato
107 jal TestOperation
108 beq a1 zero NextOperation
109 jal Add                 #' tutto corretto, posso fare la Add
110 j NextOperation

```

In seguito analizzo l'operazione Add carattere per carattere, per assicurarmi che siano presenti tutti i necessari. Dopo aver individuato una A nella lista data in input salto alla funzione TestAdd che, analogamente a WhichOperation carica il carattere che vogliamo paragonare in un registro temporaneo t0 e poi calcola la posizione dove siamo arrivati sommando i caratteri analizzati all'indirizzo s0. Poi guarda se in carattere è uguale a zero, tilde o se sono diversi ed agisce di conseguenza. Ripeto per D, D, (e).

Tra le due parentesi però devo controllare anche che il carattere racchiuso sia accettabile ovvero che sia compreso tra lo spazio e la tilde (s2 e s1), se non lo è cerco l'operazione successiva, dato che quella analizzata non è valida. Successivamente salvo il carattere contenuto nel nodo in a0.

Inizializzo un contatore per capire a che punto della lista sono e salvo il dato in a6.

A questo punto posso andare a TestOperation, che ho spiegato prima, per assicurarmi che dopo il comando ci sia una tilde, uno spazio o che la lista sia finita, altrimenti il comando non è valido. Infatti se a1=0 allora l'operazione non è scritta correttamente e devo cercare l'operazione successiva.

Se la sintassi è corretta allora possiamo entrare nella Add e una volta terminata cercherò l'operazione successiva.

```

112 Add:
113  li t0 0xFFFFFFFF        #salvo NULL in t0
114  bne s8 zero LFSR        #se s8=0 siamo nel primo nodo
115  add a1 a3 zero          #prendo l'indirizzo base
116  sw t0 0(a1)            #PBACK=NULL
117  sb a0 4(a1)            #valore contenuto nel nodo
118  sw t0 5(a1)            #PAHEAD=NULL
119  addi s8 s8 1            #contatore delle Add
120  add s4 a1 zero          #salvo l'indirizzo base del primo elemento
121  ret                    #torno a TestAdd perche' era il primo inserimento
122
123 NextAdd:
124  li t4 0xFFFFFFFF
125  sw t4 5(t2)             #PAHEAD = NULL
126  sb a0 4(t2)            #carattere da aggiungere
127  sw s4 0(t2)            #PBACK nuovo nodo punta al nodo precedente
128  sw t2 5(s4)            #PAHEAD nodo precedente punta al nuovo nodo
129  add s4 t2 zero          #nuovo indirizzo base
130  addi s8 s8 1            #contatore Add++
131  ret

```

Una volta eseguiti tutti i controlli sulla sintassi del comando dobbiamo aggiungere il nodo nella lista. Innanzitutto carico in t0 il valore NULL, poi controllo il valore di s8, un contatore di elementi della add. Se è zero allora siamo nel primo nodo, se non lo è allora dovremmo calcolare un nuovo indirizzo di memoria tramite l'LSFR (Linear Feedback Shift Register). Poi prendo l'indirizzo base a3 e lo metto in a1. Pongo il puntatore di PBACK e PAHEAD uguali a NULL cioè 0xFFFFFFFF dato che il primo nodo creato non ha nessun precedente né

successivo. Salvo anche il carattere da aggiungere in a0. Aggiorno il contatore delle Add e salvo l'indirizzo base del primo elemento in s4. Infine torno a TestAdd.

NextAdd gestisce tutte le add

successive alla prima. Come si

può intuire dalla figura per

aggiungere D' farò in modo che

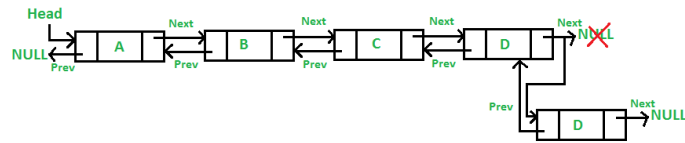
PBACK di D' punti a D e PAHEAD

di D punti a D. Infine PBACK di D'

sarà NULL.

Dunque nel codice inizio

caricando il puntatore PAHEAD uguale a NULL, dato che non ci sono altri elementi dopo il nodo che stiamo aggiungendo, poi salvo in a0 il carattere da aggiungere. Modifico PBACK facendolo puntare al nodo precedentemente aggiunto e faccio puntare PAHEAD del nodo precedente a quello aggiunto per ultimo. Salvo l'indirizzo del nuovo nodo, aggiorno il contatore degli elementi della Add e torno alla funzione che ha chiamato NextAdd.



```

133 LFSR:                                     #shift 16-k bit(k=16,14,13... seguendo il polinomio)
134     srli t2 a5 0
135     srli t4 a5 2
136     xor t2 t2 t4
137     srli t4 a5 3
138     xor t2 t2 t4
139     srli t4 a5 5
140     xor t2 t2 t4
141     slli t4 t2 15
142     srli t2 a5 1
143     or a5 t4 t2
144     li t5 0x0000FFFF
145     and t2 a5 t5
146     li t5 0x00010000
147     or t2 t2 t5                             #indirizzo casuale calcolato
148     j NextAdd

```

Quando si inserisce un nuovo nodo in una lista a puntatori, tale elemento deve prima essere memorizzato in una posizione casuale della memoria. Il concetto di casualità nell'informatica è particolarmente complesso dunque si decide di calcolare un numero casuale tramite una Pseudo Random Number Generation. In Assembly spesso si utilizza un LFSR (Linear Feedback Shift Register) che a partire da un numero di partenza espresso su n bit (seed) genera un altro numero sempre espresso su n bit, ottenuto calcolando il bit più significativo del nuovo numero calcolando un polinomio ($x^{16} + x^{14} + x^{13} + x^{11} + 1$), gli n-1 bit meno significativi si ottengono shiftando a destra il numero precedente che vengono poi messi in xor. Per il polinomio sopra si avranno quindi 4 shifts a destra: il primo di 0 (=16-16) bit, il secondo di 2 (=16-14) bit, il terzo di 3 (=16-13) bit, ed il quarto di 5 (=16-11) bit. Alla fine della funzione avremo in t2 l'indirizzo su cui salvare il nodo (che non sia il primo) e torneremo a NextAdd.

Delete:

```
150 TestDel:
151     li t0 69                #carico E e controllo
152     addi t3 t3 1
153     add t1 t3 s0
154     lb t2 0(t1)
155     beq t2 zero End
156     beq t2 s1 NextElement
157     bne t2 t0 NextOperation
158
159     li t0 76                #carico L e controllo
160     addi t3 t3 1
161     add t1 t3 s0
162     lb t2 0(t1)
163     beq t2 zero End
164     beq t2 s1 NextElement
165     bne t2 t0 NextOperation
166
167     li t0 40                #carico ( e controllo
168     addi t3 t3 1
169     add t1 t3 s0
170     lb t2 0(t1)
171     bne t2 t0 NextOperation
172
173     addi t3 t3 1
174     add t1 t3 s0
175     lb t2 0(t1)
176     blt t2 s2 NextOperation    #il carattere deve essere <126
177     bgt t2 s1 NextOperation    #il carattere deve essere >31
178     add a0 t2 zero
179
180     li t0 41                #carico ) e controllo
181     addi t3 t3 1
182     add t1 t3 s0
183     lb t2 0(t1)
184     beq t2 zero End
185     beq t2 s1 WhichOperation
186     bne t2 t0 NextOperation
187     add a6 t3 zero
188
189     jal TestOperation
190     beq a1 zero NextOperation
191     jal Del                #e' tutto corretto, posso fare la Del
192     li s3 0x00010000        #indirizzo per Del
193     add s3 t2 zero          # se non vengono effettuate delle cancellazioni, salvo in s3 nuovamente l'indirizzo Base
194     j NextOperation
```

TestDel è molto simile a TestAdd:

Analizzo l'operazione Del carattere per carattere, per assicurarmi che siano presenti tutti i necessari. Dopo aver individuato una D nella lista data in input salto alla funzione TestDel che carica il carattere che vogliamo paragonare in un registro temporaneo t0 e poi calcola la posizione dove siamo arrivati sommando i caratteri analizzati all'indirizzo s0. Poi guarda se in carattere è uguale a zero, tilde o se sono diversi ed agisce di conseguenza. Ripeto per E, L, (e).

Tra le due parentesi però devo controllare anche che il carattere racchiuso sia accettabile ovvero che sia compreso tra lo spazio e la tilde (s2 e s1), se non lo è cerco l'operazione successiva, dato che quella analizzata non è valida. Successivamente salvo il carattere contenuto nel nodo in a0.

Inizializzo un contatore per capire a che punto della lista sono e salvo il dato in a6.

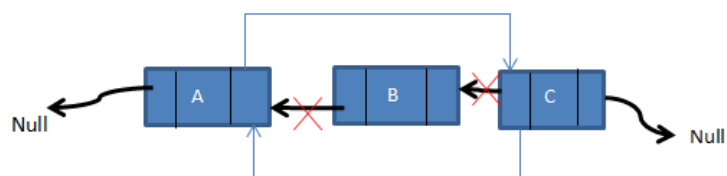
A questo punto posso andare a TestOperation, per assicurarmi che dopo il comando ci sia una tilde, uno spazio o che la lista sia finita, altrimenti il comando non è valido. Se a1=0 allora l'operazione non è scritta correttamente e devo cercare l'operazione successiva. Se la sintassi è corretta allora possiamo entrare nella Del. Salvo in s3 l'indirizzo e se non vengono fatte cancellazioni, salvo s3 nell'indirizzo base e torno a cercare un'altra operazione.

```

196 Del:
197     li t0 0xFFFFFFFF
198     beq s8 zero EndDel      #se la lista è vuota vado a EndDel
199     beq s3 t0 EndDel
200     add t6 s3 zero
201     lb t1 4(t6)             #trovo il primo carattere
202     li t4 0x00010000
203     beq t6 t4 CharTop
204
205 ContinueTop:
206     beq t6 s4 CharLast
207
208 Delete:
209     bne t1 a0 ContinueDel   #se t1 è diverso da a0 continuo a cercare
210     lw t2 0(t6)             #PBACK
211     lw t5 5(t6)             #PAHEAD
212     sw t5 5(t2)             #PAHEAD del precedente ora punta al successivo
213     sw t2 0(t5)             #PBACK del successivo ora punta al precedente
214     addi s8 s8 -1           #ho eliminato un nodo, aggiorno il contatore degli elementi aggiunti
215     j EndDel
216
217 Top:
218     lw t2 5(t6)             #PAHEAD
219     sw t0 0(t2)             #elimino il primo nodo impostando il PBACK del secondo uguale a NULL
220     add s3 t2 zero          #s3 nuovo top
221     ret
222
223 Last:
224     lw t2 0(t6)
225     sw t0 5(t2)             #PAHEAD penultimo NULL
226     add s4 t2 zero          #salvo il nuovo ultimo
227     addi s8 s8 -1
228     j EndDel
229
230 ContinueDel:
231     lw t6 5(t6)             #scorro all'elemento successivo
232     add s3 t6 zero
233     j Del
234
235 CharTop: #trovo il carattere da eliminare in cima
236     beq t1 a0 Top
237     j ContinueTop
238
239 CharLast: #trovo il carattere da eliminare in fondo
240     beq t1 a0 Last
241     j Delete
242
243 EndDel:
244     li t2 0x00010000
245     ret

```

Il comando Del serve a cancellare logicamente un nodo. Questo vuol dire che il nodo non verrà



effettivamente eliminato, bensì modificando i puntatori, lo renderemo irraggiungibile. Per esempio, dati i nodi A, B e C, è possibile cancellare logicamente il nodo B facendo in modo che il puntatore PAHEAD di A corrisponda a PBACK di C e non più a quello di B, rendendo quest'ultimo irraggiungibile.

In questa funzione carico in t0 il valore NULL, poi controllo il valore di s8, un contatore di elementi della add per verificare che la lista sia vuota e nel caso vado a EndDel. Poi controllo se l'indirizzo in s3 è NULL, poiché significherebbe che non è stata effettuata nessuna cancellazione. Anche in quel caso vado in EndDel.

Calcolo in t6 il carattere presente all'indirizzo s3 e lo carico in t1. Inseguito verifico che l'indirizzo in t6 sia uguale a t4, l'indirizzo del primo nodo della catena e vado a Top.

Altrimenti trovo ContinueTop che mi porta all'ultimo carattere aggiunto.

La funzione che cancella logicamente il nodo è Delete che controlla che il carattere da eliminare corrisponda al carattere trovato nel TestDel. Se non sono uguali vado a ContinueDel, altrimenti procedo con la cancellazione logica. Carico i due puntatori PBACK e PAHEAD. Faccio poi in modo che PAHEAD del precedente punti al successivo e PBACK del successivo punti al precedente, come spiegato con l'esempio dei nodi A, B e C. A questo punto sarà impossibile raggiungere il nodo eliminato e dunque aggiorno il contatore del numero di caratteri in s8 diminuendolo di 1 e saltando a EndDel.

In seguito troviamo Top, utilizzato per la cancellazione del primo elemento della lista. Per questo infatti basterà porre PBACK del secondo elemento s3 uguale a NULL.

Per Last invece pongo PAHEAD del penultimo elemento uguale a NULL, in modo da eliminare l'ultimo elemento.

Se in Delete l'indirizzo in t1 non equivale a a0, l'indirizzo in cima alla lista concatenata passo alla funzione ContinueDel che scorre all'elemento successivo della lista.

Sort:

```
247 TestSort:
248     li t0 79                                #carico 0 e controllo
249     addi t3 t3 1
250     add t1 t3 s0
251     lb t2 0(t1)
252     beq t2 zero End
253     beq t2 s1 WhichOperation
254     bne t2 t0 WhichOperation
255
256     li t0 82                                #carico R e controllo
257     addi t3 t3 1
258     add t1 t3 s0
259     lb t2 0(t1)
260     beq t2 zero End
261     beq t2 s1 WhichOperation
262     bne t2 t0 WhichOperation
263
264     li t0 84                                #carico T e controllo
265     addi t3 t3 1
266     add t1 t3 s0
267     lb t2 0(t1)
268     beq t2 zero End
269     beq t2 s1 WhichOperation
270     bne t2 t0 WhichOperation
271
272     add a6 t3 zero
273     jal TestOperation
274     beq a1 zero NextOperation
275     jal Sort                                #e' tutto corretto, posso fare il Sort
276     j NextOperation
```

TestSort controlla la sintassi dell'operazione analogamente a TestAdd e TestDel. Controlla via via i caratteri O, R e T assicurandosi che non sia finita la lista o che sia presente una tilde. In tal caso la funzione richiamerà rispettivamente End o WhichOperation. Se t2, il carattere nella posizione calcolata non è uguale al carattere caricato allora cerchiamo la prossima operazione valida, altrimenti continuiamo con i test. Nell'ultima parte richiamiamo TestOperation per analizzare i caratteri successivi. Se l'operazione è scritta correttamente possiamo passare al Sort.

```
278 Sort:
279     li t4 0                #contatore per scambi
280     add a3 s3 zero
281     lb t1 4(a3)            #prendo il carattere
282     blt t1 s2 EndSort      #carattere <32 EndSort
283     li s2 2
284     blt s8 s2 EndSort      #n.elementi nella lista <2 EndSort
285
286 IsCapital:
287     li t0 65
288     blt t1 t0 IsNumber
289     li t0 90
290     bgt t1 t0 IsLowercase
291     addi t1 t1 97
292     addi s10 s10 1
293     j NextNodes
294
295 IsLowercase:
296     li t0 97
297     blt t1 t0 IsCharExtra2
298     li t0 122
299     bgt t1 t0 IsCharExtra
300     addi t1 t1 39
301     addi s10 s10 1
302     j NextNodes
303
304 IsNumber:
305     li t0 48
306     blt t1 t0 IsCharExtra
307     li t0 57
308     bgt t1 t0 IsCharExtra1
309     addi t1 t1 78
310     addi s10 s10 1
311     j NextNodes
```

```

313 CharExtra:                                #Racchiude tutte le altre categorie di caratteri EXTRA la di fuori delle sottostanti
314     addi s10 s10 1
315     j NextNodes
316
317 CharExtra1:                                #caratteri compresi tra 57 e 65
318     li t0 65
319     blt t1 t0 CharExtra
320
321 CharExtra2:                                #caratteri compresi tra 90 e 97
322     li t0 90
323     bgt t1 t0 CharExtra
324
325 Tests:
326     li s6 0xFFFFFFFF
327     beq a6 s6 Again                        #fine lista
328     blt t1 s2 EndSort                     #carattere <32
329
330 Capital:
331     li t0 65
332     blt t1 t0 Numbers
333     li t0 90
334     bgt t1 t0 Lowercase
335     addi t1 t1 97
336     addi s10 s10 1
337     j NextNodes
338
339 Lowercase:
340     li t0 97
341     blt t1 t0 Extra2
342     li t0 122
343     bgt t1 t0 Extra
344     addi t1 t1 39
345     addi s10 s10 1
346     j NextNodes
347
348 Numbers:
349     li t0 48
350     blt t1 t0 Extra
351     li t0 57
352     bgt t1 t0 Extra1
353     addi t1 t1 78
354     addi s10 s10 1
355     j NextNodes
356
357 Extra:                                    #caratteri Extra oltre a quelli già visti
358     addi s10 s10 1
359     j NextNodes
360
361 Extra1:                                    #caratteri compresi tra 57 e 65
362     li t0 65
363     blt t1 t0 Extra
364
365 Extra2:                                    #caratteri compresi tra 90 e 97
366     li t0 90
367     bgt t1 t0 Extra
368
369 NextNodes:
370     li t0 2
371     beq s10 t0 BubbleSort                 #se ho gi? trovato 2 nodi faccio il Sort
372     add t5 t1 zero                         #primo nodo
373     lw a6 5(a3)
374     lb t1 4(a6)                           #carico il successivo
375     j Tests

```

```

377 Again:
378     li a1 0
379     beq t4 a1 EndSort
380     li t4 0
381     add s10 zero zero
382     add a6 zero zero
383     j Sort
384
385 BubbleSort:
386     bgt t5 t1 Swap
387     lw a3 5(a3)
388     lb t1 4(a3)
389     addi s10 s10 -2
390     j Tests
391
392 Swap:
393     addi t4 t4 1
394     lb t2 4(a3)      #carico i caratteri
395     lb t6 4(a6)
396     sb t2 4(a6)      #salvo i caratteri invertendoli di posizione
397     sb t6 4(a3)
398     lw a3 5(a3)      #nodo successivo
399     lb t1 4(a3)      #salvo il carattere
400     addi s10 s10 -2
401     j Tests
402
403 EndSort:
404     ret

```

Il Sort da applicare sui caratteri ASCII segue il seguente criterio di ordinamento transitivo:

- una lettera maiuscola (ASCII da 65 a 90 compresi) viene sempre ritenuta maggiore di una minuscola
- una lettera minuscola (ASCII da 97 a 122 compresi) viene sempre ritenuta maggiore di un numero
- un numero (ASCII da 48 a 57 compresi) viene sempre ritenuto maggiore di un carattere extra che non sia lettera o numero
- non si considerano accettabili caratteri extra con codice ASCII minore di 32.

Inoltre, all'interno di ogni categoria vige l'ordinamento dato dal codice ASCII. Per esempio, date due lettere maiuscole x e x' , $x < x'$ se e solo se $\text{ASCII}(x) < \text{ASCII}(x')$. Lo stesso vale per le lettere minuscole, per i numeri e per i caratteri extra.

Ad esempio, la sequenza a.2Er4,w si ordinerà come ,.24arwE

Il codice ASCII segue l'ordinamento

1. caratteri extra (32-47)
2. numeri (48-57)
3. maiuscole (65-90)
4. minuscole (97-122)

per riuscire ad aggirare questo problema ho aggiunto ad ogni categoria una costante in modo da ottenere un ordinamento coerente con le richieste.

1. caratteri extra (32-47) $\rightarrow +0 \rightarrow 1.$ (32-47)
2. numeri (48-57) $\rightarrow +78 \rightarrow 2.$ (126-135)
3. maiuscole (65-90) $\rightarrow +97 \rightarrow 4.$ (162-187)
4. minuscole (97-122) $\rightarrow +39 \rightarrow 3.$ (136-161)

Nella funzione Sort inizializzo un registro t4 che mi servirà da contatore si scambi nel BubbleSort. Calcolo l'indirizzo in a3 l'indirizzo che contiene la testa della lista e la carico nel registro t1 il carattere contenuto in quell'indirizzo.

Controllo che il primo elemento della lista non sia un valore <32, poiché inaccettabile e verifico che il numero di elementi presenti da ordinare non sia minore di 2, In tal caso, con un solo elemento la lista sarebbe già ordinata.

Successivamente inizio a controllare il carattere per vedere se è una lettera maiuscola, minuscola, un numero o un carattere extra.

In IsCapital carico i due valori massimi e minimi accettabili. Se il carattere analizzato è minore di 65 vado a IsNumber, poi carico 90 e se maggiore, vado a IsLowercase. Ora che sono sicura di avere un carattere maiuscolo aggiungo a t1 97, aggiorno il contatore e vado a NextNodes.

In IsLowercase carico 97. Se il carattere è minore di 97 allora sarà in carattere speciale del tipo Extra2 [] ^ _ , altrimenti se maggiore di 122 sarà un carattere Extra1 { | } . Ora che sono sicura che sia una minuscola posso aggiungere a t1 39, aggiorno il contatore e vado a NextNodes.

In IsNumber carico 48. Se il carattere è minore allora sarà un carattere del tipo Extra ! " # ' () * + , altrimenti se maggiore di 57 sarà del tipo Extra1. Ora che sono sicura che sia un numero posso aggiungere a t1 78, aggiorno il contatore e vado a NextNodes.

IsCharExtra aggiorna il contatore e va a next nodes, stessa cosa per IsCharExtra1 e IsCharExtra2 che vengono reindirizzati a isCharExtra.

In Tests controllo che il carattere a6 sia uguale a NULL e in tal caso vado a Again. Poi controllo anche che il carattere sia maggiore di 32, lo spazio altrimenti il valore è inaccettabile e vado a EndSort.

Dopo questi primi test ricerco a quale categoria appartiene il carattere in maniera del tutto analoga a quella appena vista.

In seguito arriviamo a NextNodes. Controlliamo 2 caratteri della lista concatenata si entra nel BubbleSort, altrimenti vado a salvare il carattere analizzato in precedenza in t5 e passo al nodo successivo memorizzando il carattere in t1.

In Again verifico se sono a fine lista. Se il contatore t4 è a 0 allora il Sort può terminare.

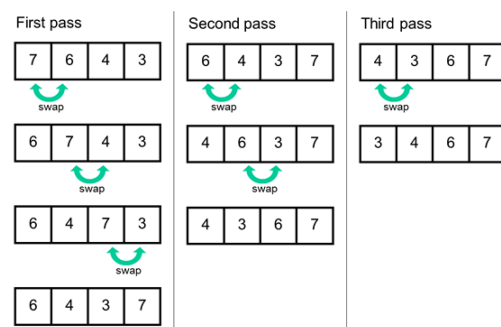
La tipologia di ordinamento che utilizzo è il BubbleSort. Esso consiste nell'ordinare ogni coppia di elementi adiacenti, che vengono comparati ed invertiti di posizione se sono nell'ordine sbagliato.

L'algoritmo continua a ri-eseguire questi passaggi su tutta la lista fino a quando non vengono più eseguiti scambi poiché la lista è ordinata

In Bubblesort se t5 è maggiore t1 vado in Swap, altrimenti passo al nodo

successivo di a3 e memorizzo il carattere in t1 e decremento s10 di 2.

In Swap incremento il contatore degli scambi t4 di 1 carico i due caratteri presenti negli indirizzi di a3 e a6, rispettivamente in t2 e t6 per poterli scambiare. Poi passo al nodo successivo con registro a3, memorizzo il carattere in t1, passo al nodo successivo di a6 e decremento di 2 s10 e salto a Tests.



Reverse:

```
406 TestRev:
407     li t0 69                                #carico E e controllo
408     addi t3 t3 1
409     add t1 t3 s0
410     lb t2 0(t1)
411     beq t2 zero End
412     beq t2 s1 NextElement
413     bne t2 t0 NextOperation
414
415     li t0 86                                #carico V e controllo
416     addi t3 t3 1
417     add t1 t3 s0
418     lb t2 0(t1)
419     beq t2 zero End
420     beq t2 s1 NextElement
421     bne t2 t0 NextOperation
422
423     add a6 t3 zero
424     jal TestOperation
425     beq a1 zero NextOperation
426     jal Rev                                #e' tutto corretto, posso fare la Rev
427     j NextOperation
```

TestRev controlla la sintassi dell'operazione in maniera molto simile a TestSort. Controlla via via i caratteri E e V assicurandosi che non sia finita la lista o che sia presente una tilde. In tal caso la funzione richiamerà rispettivamente End o WhichOperation. Se t2, il carattere nella posizione calcolata non è uguale al carattere caricato allora cerchiamo la prossima operazione valida, altrimenti continuiamo con i test. Nell'ultima parte richiamiamo TestOperation per analizzare i caratteri successivi. Se l'operazione è scritta correttamente possiamo passare alla Rev.

```

429 Rev:
430     li t6 2
431     blt s8 t6 EndRev
432     add t5 s3 zero
433     add t6 s4 zero
434     lb t0 4(t5)           #carico il carattere in testa in t0
435     lb t1 4(t6)           #carico il carattere in coda in t1
436     sb t0 4(t6)           #metto t0 coda
437     sb t1 4(t5)           #metto t1 in testa
438
439 LoopRev:
440     lw t5 5(t5)           #scorro dalla testa fino alla coda
441     lw t6 0(t6)           #scorro dalla coda alla testa
442     beq t5 t6 EndRev      #puntano allo stesso elemento, esco
443     lw t4 0(t5)
444     beq t4 t6 EndRev      #se si incrociano sono pari, esco
445     lb t0 4(t5)
446     lb t1 4(t6)
447     sb t0 4(t6)
448     sb t1 4(t5)          #scambio gli elementi
449     j LoopRev
450
451 EndRev:
452     ret

```

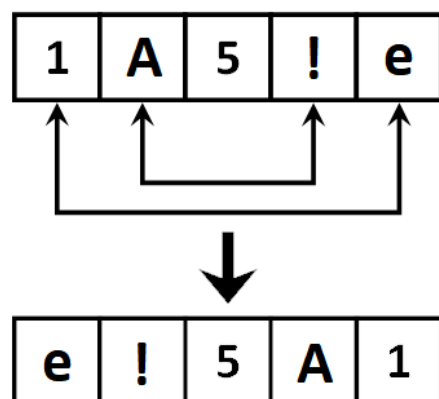
Il comando Rev inverte gli elementi della nostra lista data in input.

Controllo subito che la lista non sia costituita da meno di due elementi, infatti sarebbe inutile svolgere la Rev. Poi mi calcolo gli indirizzi del primo e dell'ultimo nodo, carico il carattere presente in testa in t0 e il carattere in coda in t1, e inverte le loro posizioni. Subito dopo entro in

LoopRev e inizio a scorrere i nodi sia dall'inizio che dalla fine della lista. Se i due registri t5 e t6 sono uguali allora puntano al solito carattere, la Rev è terminata e posso uscire. Questo succede nelle liste con un numero di caratteri dispari.

Altrimenti continuo finché t5 e t6 non si incrociano, a quel punto avrò invertito tutti gli elementi di una lista con un elementi pari, quindi posso uscire dalla Rev. Se non siamo ancora arrivati alla fine scambio

le posizioni degli elementi caricati in t5 e t6 e riparto con il loop.



Print:

```
454 TestPrint:
455     li t0 82                #carico R e controllo
456     addi t3 t3 1
457     add t1 t3 s0
458     lb t2 0(t1)
459     beq t2 zero End
460     beq t2 s1 NextElement
461     bne t2 t0 NextOperation
462
463     li t0 73                #carico I e controllo
464     addi t3 t3 1
465     add t1 t3 s0
466     lb t2 0(t1)
467     beq t2 zero End
468     beq t2 s1 NextElement
469     beq t2 s1 WhichOperation
470
471     li t0 78                #carico N e controllo
472     addi t3 t3 1
473     add t1 t3 s0
474     lb t2 0(t1)
475     beq t2 zero End
476     beq t2 s1 NextElement
477     beq t2 s1 WhichOperation
478
479     li t0 84                #carico T e controllo
480     addi t3 t3 1
481     add t1 t3 s0
482     lb t2 0(t1)
483     beq t2 zero End
484     beq t2 s1 NextElement
485     beq t2 s1 WhichOperation
486
487     add a6 t3 zero
488     jal TestOperation
489     beq a1 zero NextOperation
490     jal Print                #'e' tutto corretto, posso fare la Print
491     j NextOperation
492
493 Print: #scorro tutti gli elementi finche' non trovo NULL
494     li s3 0x00010000        #indirizzo per Print
495     li t2 0xFFFFFFFF
496     beq s3 t2 EndPrint
497     add a4 s3 zero
498
499 LoopPrint:
500     beq a4 t2 EndPrint
501     lb a0 4(a4)              #a0 e' il carattere da stampare
502     li a7 11
503     ecall
504     lw a4 5(a4)
505     j LoopPrint
506
507 EndPrint:
508     la a0, space
509     li a7,4
510     ecall
511     ret
512
513 End:
514     add a7 a7 zero
```


TestPrint controlla la sintassi dell'operazione in maniera molto simile a TestSort. Controlla via via i caratteri I, N e T assicurandosi che non sia finita la lista o che sia presente una tilde. In tal caso la funzione richiamerà rispettivamente End o WhichOperation. Se t2, il carattere nella posizione calcolata non è uguale al carattere caricato allora cerchiamo la prossima operazione valida, altrimenti continuiamo con i test. Nell'ultima parte richiamiamo TestOperation per analizzare i caratteri successivi. Se l'operazione è scritta correttamente possiamo passare alla Print.

Nella Print stampa tutti i caratteri presenti nei nodi della lista.

Se l'indirizzo da cui parto è NULL allora vado ad EndPrint. Poi mi calcolo l'indirizzo necessario in a4 ed entro nel LoopPrint.

Come sempre, se a4 è NULL, esco dal loop e vado in EndPrint.

Carico il primo carattere che mi interessa in a0 e lo stampo grazie all'ecall, poi incremento il registro a4 in modo che ci sia caricato l'indirizzo del nodo successivo. Continuo finché l'indirizzo del nodo successivo non è NULL e quindi ho terminato la mia lista.

In EndPrint carico in a0 la stringa space per riuscire a intervallare i caratteri ed avere un'output più chiaro.

Test:

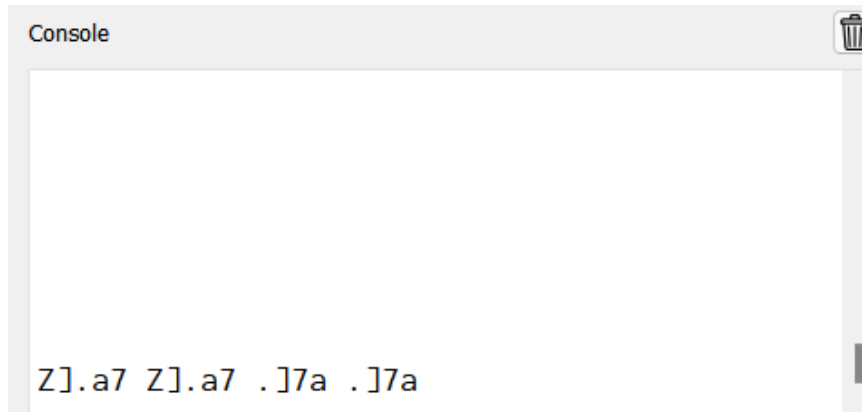
```
listInput = "ADD(1) ~ ADD(a) ~ ADD(a) ~ ADD(B) ~ ADD(;) ~ ADD(9)  
~PRINT~SORT~PRINT~DEL(b) ~DEL(B)~PRI~REV~PRINT"
```

```
Console  
1aaB;9 ;19aaB aa91;
```

```
listInput = "ADD(1) ~ ADD(a) ~ ADD() ~ ADD(B) ~ ADD ~ ADD(9)  
~PRINT~SORT(a)~PRINT~DEL(bb) ~DEL(B)~PRINT~REV~PRINT"
```

```
Console  
1aB9 1aB9 1a9 9a1
```

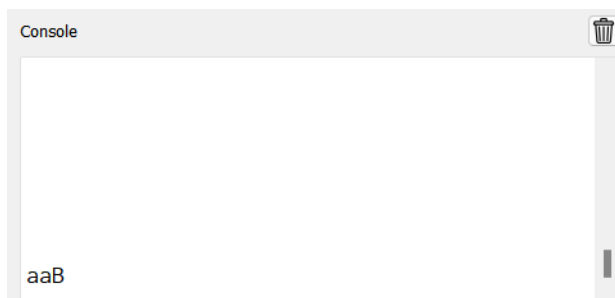
```
listInput="ADD(Z) ~ ADD(I) ~ ADD(.) ~ ADD(a) ~ AD ~ ADD(7)  
~PRINT~SORT(~PRINT~SORT~DEL(Z)~DEL(K)~PRINT~REV(~PRINT"
```



Come richiesto, il programma esegue le operazioni solo se scritte correttamente.

- Nel caso delle operazioni ADD e DEL si suppone di avere uno ed uno solo carattere tra parentesi; Nel caso in cui compaiano zero o più di un carattere tra parentesi, l'operazione si considera mal formattata e da scartare.

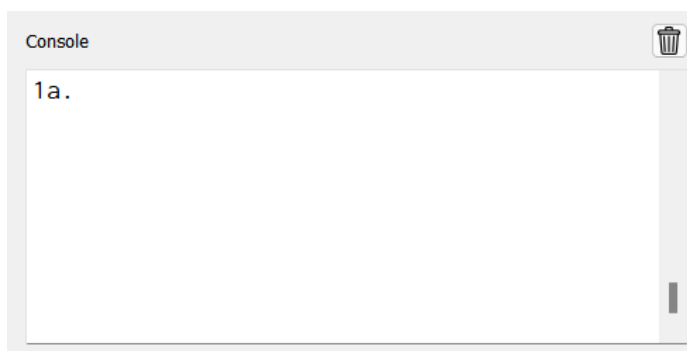
Esempio "ADD(11) ~ ADD(a) ~ ADD(a) ~ ADD(B)~PRINT"



ADD(11) viene scartata

- I caratteri dei comandi devono essere consecutivi: sarà ammissibile il comando "SORT" ma non il "SO RT". Allo stesso modo, è ammissibile "ADD(d)" ma non "AD D(d)" o "ADD (d)"

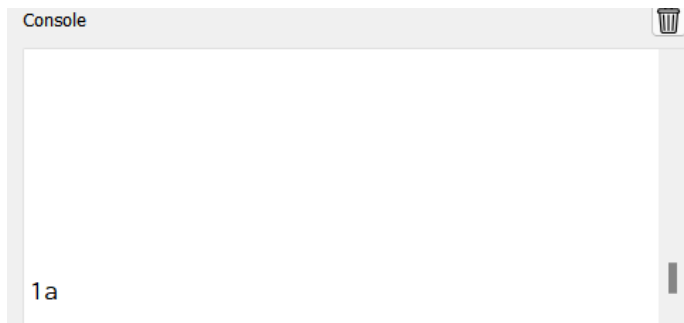
Esempio: "ADD(1) ~ AD D(a) ~ ADD(a) ~SORT~ADD(.)~SO RT~PRINT"



AD D(a) e SO RT non sono considerati accettabili

- Spazi vicini alle ~ sono ammessi e devono essere tollerati dal programma. (Si può notare dagli esempi precedenti)

- Il comando deve essere espresso con lettere maiuscole. "PRINT" è ammissibile, "print" non lo è
Esempio: "ADD(1) ~ ADD(a) ~print~PRINT"



La stampa viene eseguita una sola volta, print non è accettabile